

Design of Experiments (DoE)

Modelització Estadística

Grau en Intel·ligència Artificial

Course 2024-25



Outline

- 1 Introduction
- 2 Interactions
- 3 Types of Experimental Designs
- 4 Statistical Analysis
- 5 References

Introduction to Design of Experiments (DOE)

What is DOE?

Design of Experiments (**DOE**) is a structured methodology for planning and conducting experiments efficiently:

- **Objective:** Identify the factors that **influence an outcome** and optimize performance.
- **Why Use DOE?:**
 - Reduces experimentation time and cost.
 - Provides insights into interactions between variables.
 - Ensures reliable conclusions by controlling for randomness.

Why Experiment?

- **Causality:** Experiments reveal cause-and-effect relationships, unlike observational studies.
- **Control:** Allows systematic manipulation of factors to isolate their effects.
- **Insights:** Helps prioritize variables with the highest impact on outcomes.

Example

Example

We want to improve our knowledge in a machine learning process:

- **Feature Engineering:** Test the impact of different feature transformations on model performance.
- **Algorithm Tuning:** Compare algorithms under controlled conditions to select the best one for a task.
- **Hyperparameter Optimization:** Identify the best combinations of parameters (e.g., learning rate, batch size).

Factors to test:

- **Preprocessing:** Standardization vs. normalization.
- **Algorithm:** Logistic regression vs. random forest.
- **Hyperparameters:** Learning rate (low vs. medium vs. high).

A DOE approach assesses these combinations efficiently, revealing the best setup.

Factors and Levels

What are Factors and Levels?

- **Factors:** Variables that can influence the outcome of an experiment.
 - Examples in AI: Learning rate, number of layers, optimizer type.
- **Levels:** The specific values or settings that a factor can take.
 - Examples: Low, Medium, High for learning rate.

Types of Factors

- **Categorical Factors:**
 - Discrete options or categories.
 - Example: Optimizer type (Adam, SGD, RMSprop).
- **Continuous Factors:**
 - Numeric variables with a range of possible values.
 - Example: Learning rate (0.001, 0.01, 0.1).

Remark

A careful **selection of factors and levels** is crucial for meaningful experimentation.

Example

Context: AI Pipeline Optimization

- Factors:
 - **Data Augmentation:** Categorical (Yes, No).
 - **Batch Size:** Continuous (16, 32, 64).
- Levels:
 - **Data Augmentation:**
 - Level 1: Yes.
 - Level 2: No.
 - **Batch Size:**
 - Level 1: 16.
 - Level 2: 32.
 - Level 3: 64.
- Combining these factors and levels creates an experimental design.

Outcome

Outcome

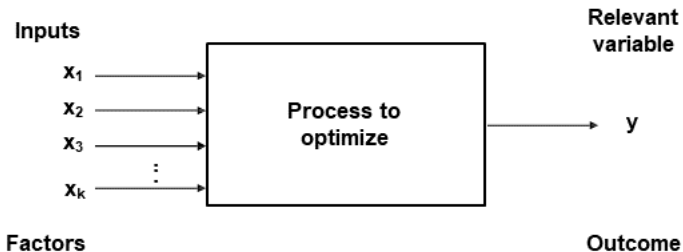
In DOE, the **outcome** is the variable to optimize during the experiment:

- Represent the results of **interest** influenced by the factors.
- Outcomes are analyzed to understand the **effects** of factors and their **interactions**.

Example

- **Problem**: Optimizing a neural network model.
- **Outcomes**:
 - **Accuracy**: Percentage of correct predictions on the test set.
 - **Training Time**: Total time taken to complete training.
- **Goal**: Maximize accuracy constrained to a fixed training time.

General schema



Simulations. Pros and cons

- An alternative to perform experiments is to simulate data.

Pros

Simulations can reduce the standard error to 0 (not the standard deviation).

- Despite the usefulness of simulations, some situations require experimentation.

Cons

The main challenge lies in systematic errors or inaccuracies in model specification.

- We **lack the knowledge** to create accurate models or face situations that are complex to model.
- We **do not have empirical data** to estimate certain parameters.
- The models we assume to be true **are not accurate** representations of reality.

The Need of Experimentation

While experimentation is necessary for addressing uncertainties and validating models, it comes with challenges:

- **Experimentation is expensive:** Both in terms of resources and time.
- **Balancing Simulations and Experiments:**
 - Use simulations to refine hypotheses and reduce the experimental space.
 - Combine both approaches to achieve reliable and cost-effective results.

Interactions

What are Interactions?

- **Interactions** occur when the effect of one factor depends on the level of another factor.
- They reveal how factors work together rather than independently.

Example

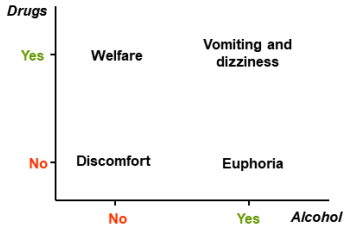
The impact of **learning rate** on model accuracy might depend on the **batch size**.

Why Are Interactions Important?

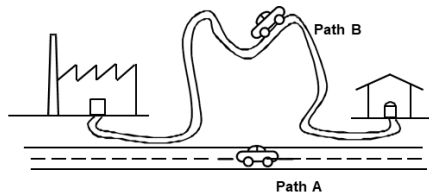
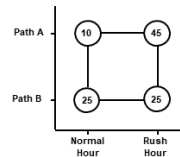
- **Understanding interactions** helps identify effects between factors.
- Ignoring interactions can lead to **misleading conclusions**.
- Interaction effects can guide **fine-tuning of AI pipelines**.

Interactions. Examples

Drugs vs. Alcohol



Path vs. Time slot



Synergistic and Antagonistic Factors

Synergistic Factors

Two factors are **synergistic** when the effect of applying them together is **greater** than the sum of their individual effects.

- An increase in pressure alone results in a gain of **+5** in the product, and an increase in temperature alone results in a gain of **+5**. An increase in both results in a gain of **+15**.

Antagonistic Factors

Two factors are **antagonistic** when the effect of applying them together is **less** than the sum of their individual effects.

- An increase in pressure alone results in a gain of **+5** in the product, and an increase in temperature alone results in a gain of **+5**. However, an increase in both results in a gain of **+8**.

Remark

To discuss synergistic or antagonistic factors, **reference levels** must be established, such as the *absence of...* a condition.

Visualize interactions

Example

- **Problem:** Evaluate the combined effect of learning rate and batch size on model accuracy.
- **Visualization:** `interaction.plot` in R
- **Interpretation:**
 - **Parallel lines** indicate no interaction.
 - **Diverging lines** suggest strong interaction effects.

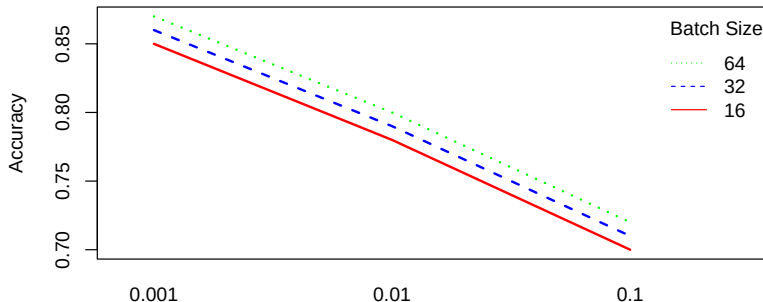
Visualize interactions. Data without interaction

- **Factors:** Learning rate, Batch size
- **Outcome:** Accuracy

##	learning_rate	batch_size	accuracy
## 1	0.001	16	0.85
## 2	0.001	32	0.86
## 3	0.001	64	0.87
## 4	0.010	16	0.78
## 5	0.010	32	0.79
## 6	0.010	64	0.80
## 7	0.100	16	0.70
## 8	0.100	32	0.71
## 9	0.100	64	0.72

Visualize interactions. Interaction Plot (NO interaction)

```
interaction.plot(x.factor      = data$learning_rate,
                 trace.factor  = data$batch_size,
                 response       = data$accuracy,
                 xlab = "Learning Rate", ylab = "Accuracy",
                 trace.label = "Batch Size", lwd=2, lty = 1:3,
                 col = c("red", "blue", "green"), pch = 16:18)
```



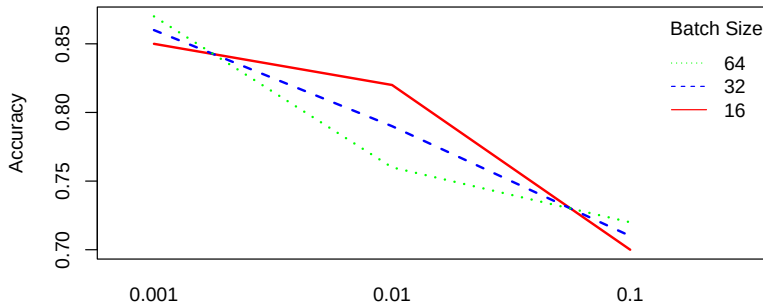
Visualize interactions. Data with interaction

- **Factors:** Learning rate, Batch size
- **Outcome:** Accuracy

##	learning_rate	batch_size	accuracy
## 1	0.001	16	0.85
## 2	0.001	32	0.86
## 3	0.001	64	0.87
## 4	0.010	16	0.78
## 5	0.010	32	0.79
## 6	0.010	64	0.80
## 7	0.100	16	0.70
## 8	0.100	32	0.71
## 9	0.100	64	0.72

Visualize interactions. Interaction Plot (Interaction)

```
interaction.plot(x.factor      = data2$learning_rate,
                 trace.factor  = data2$batch_size,
                 response      = data2$accuracy,
                 xlab = "Learning Rate", ylab = "Accuracy",
                 trace.label = "Batch Size", lwd=2, lty = 1:3,
                 col = c("red", "blue", "green"), pch = 16:18)
```



Strategies. Using existing data

Alternative to Experimentation: Old data

The first alternative to save on experimentation costs could be to use already existing data.

- If the data are **observational** (not from an experiment), we cannot establish causality.
 - **Example:** Did the process deteriorate in February because we changed Machine A, or because Employee B left?
- If the data come from an **experiment**, the main issue is that the data might be inconsistent due to changes over time:
 - **Aging:** Components or materials may degrade over time.
 - **Repairs:** Equipment adjustments can introduce variability.
 - **Procedure Changes:** Modifications in the process may affect results.

Strategies. Plan

Experimenting Without a Plan

Making changes to the system and observing how productivity changes.

- This approach is **not recommended**.

Designing the Entire Plan Upfront

- If we know the system well, this can be an alternative.
- Otherwise, it is not advisable to invest the entire budget in a single experiment.
 - There are many things that could go wrong!

Sequential Strategy

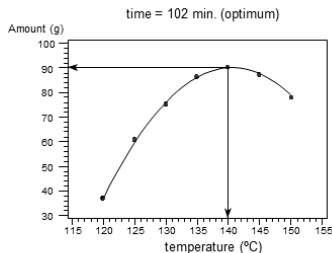
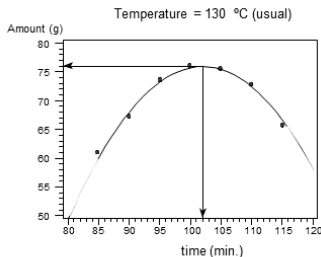
This is the **preferred strategy**.

- A first phase can be carried out with real experimentation or simulation.
- A second phase can follow with real experimentation.

Strategies. Naive strategy

Naive strategy

- ① It consists of fixing the value of all factors except one, which will be varied.
- ② Find the optimal value of the unfixed factor and assign it that value.
- ③ Repeat the same process with the remaining factors.

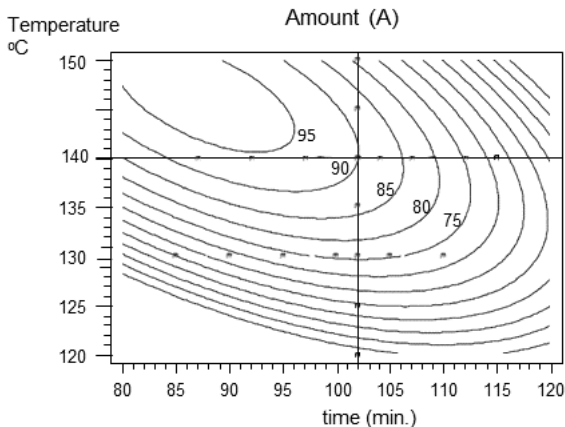


Optimum:

T = 140 °C ; t = 102 min → A = 90 g

¿Correct?

Strategies. Naive strategy. Problem



Problem of Naive Strategy

If there are interactions, the **naive strategy does not properly work**

Outline: Types of designs

- 1 Full Factorial Design (FFD)
- 2 Fractional Factorial Designs (FD)
- 3 Latin Squares (LS)
- 4 Optimal Designs

Full Factorial Design

What is a Full Factorial Design?

A **full factorial design** (FFD) is an experimental setup where all possible combinations of factor levels are tested:

- **Objective:** Study the effect of each factor and their interactions on the outcome.
- **Structure:**
 - k factors with l_i levels ($i = 1, 2, \dots, k$).
 - Total combinations: $\prod_{i=1}^k l_i$.
- Suitable for understanding:
 - **Main effects** of factors.
 - **All** interactions between factors.

Example

- Factors:
 - **Learning Rate** ($l_1 = 2$): 0.01, 0.1.
 - **Batch Size** ($l_2 = 3$): 16, 32, 64.
- Total combinations: $2 \times 3 = 6$.
- Experimental layout:

Learning Rate	Batch Size
0.01	16
0.01	32
0.01	64
0.1	16
0.1	32
0.1	64

Pros of FFD

- **Thorough Analysis:**
 - Identifies main effects and all interaction effects.
- **Accurate Insights:**
 - Ensures no important effects are overlooked.
- **Applicable to Small Designs:**
 - Useful when the number of factors and levels is manageable.

Cons of FFD

- **Exponential Growth of Combinations:**
 - Adding factors or levels significantly increases the total runs.
- **Resource Intensive:**
 - Time and computational costs grow quickly with the number of combinations.
- **Cost:**
 - Expensive in real situations.
- **Complex Analysis for Many Factors:**
 - High-dimensional data can be challenging to interpret.

R

Use the DoE.base package to generate a full factorial design.

```
library(DoE.base)
# Define factors and levels
factors <- list(learning_rate = c(0.01, 0.1),
               batch_size      = c(16, 32, 64))
# Generate design
full_design <- fac.design(nlevels=c(2,3),nfactors = 2,
                          factor.names = names(factors))
full_design # Combinations are intentionally unsorted
```

```
##   learning_rate batch_size
## 1             1           2
## 2             2           3
## 3             2           1
## 4             2           2
## 5             1           3
## 6             1           1
## class=design, type= full factorial
```

Design matrix

What is a Design Matrix?

- A **design matrix** is a structured representation of all the experimental runs in a factorial experiment.
- It shows the levels of each factor for every experimental combination.
- In the context of a two-level factorial design, the levels are coded as:
 - **+1**: Represents the high or maximum level of a factor.
 - **-1**: Represents the low or minimum level of a factor.

Benefits of Using Coded Levels (-1 and +1)

- Simplifies calculations for estimating effects and interactions.
- Ensures all factors are on the same scale, avoiding unit mismatch.
- Enables easier interpretation of factor effects.

Example Design Matrix for Two Factors

Run	Factor A	Factor B
1	-1	-1
2	-1	+1
3	+1	-1
4	+1	+1

Fractional Factorial Design

What is a Fractional Factorial Design?

A **fractional factorial design** is a reduced version of a full factorial design:

- **Objective:** Test a carefully selected subset of all possible factor-level combinations.
- **Why Use It?**
 - Reduces the number of experiments while retaining critical information.
 - Useful when resources are limited, but key effects still need to be estimated.

Key Features of FD Construction

- The goal is to estimate the **maximum number of effects** while omitting unnecessary combinations.
- Inevitably, some effects will be **confounded** (cannot be distinguished from one another).
- Main effects should ideally be confounded with interactions of the **highest order possible**.

Orthogonality

What is Orthogonality?

- Orthogonality refers to the relationship between two vectors in a vector space. Two vectors are said to be orthogonal if their **dot product is zero**.
- A fractional factorial design (FD) omits some combinations of factor levels while attempting to maintain **orthogonality**.
- Orthogonality **ensures independence among the estimated effects**, enabling clear interpretation of the results:
 - The estimated coefficients are the same regardless the factors included in the model.

Fractional Factorial Designs construction. Generators

Generators

- To construct a FD from a FFD, a **generator** is needed.
- The choice of the generator considers the **principles of orthogonality and confounding**.
 - **Orthogonality** is guarantee in a FD.
 - A FD ensures main effects are **confounded** with the highest-order interactions possible.
- A generator is a **product of columns** of the design matrix that serves to generate a new column.
- In general, how to choose a generator **is not trivial**.
- For instance, for constructing a fractional factorial design (2^4) from a full factorial design (2^5):
 - A full factorial design for 4 factors (A, B, C, D) is constructed first.
 - Factor E is defined using the generator $E = ABCD$, where
 - The value of E is calculated by multiplying the levels of A, B, C , and D (e.g., $E = (+1)(-1)(+1)(+1) = -1$).

Example

- We need 1 generator to go from 16 to 8 experiments.

Prod.	A	B	C	D	E
1	-1	-1	-1	-1	-1
2	1	-1	-1	-1	-1
3	-1	1	-1	-1	-1
4	1	1	-1	-1	-1
5	-1	-1	1	-1	-1
6	1	-1	1	-1	-1
7	-1	1	1	-1	-1
8	1	1	1	-1	-1
9	-1	-1	-1	1	-1
10	1	-1	-1	1	-1
11	-1	1	-1	1	-1
12	1	1	-1	1	-1
13	-1	-1	1	1	-1
14	1	-1	1	1	-1
15	-1	1	1	1	-1
16	1	1	1	1	-1
17	-1	-1	-1	-1	1
18	1	-1	-1	-1	1
19	-1	1	-1	-1	1
20	1	1	-1	-1	1
21	-1	-1	1	-1	1
22	1	-1	1	-1	1
23	-1	1	1	-1	1
24	1	1	1	-1	1
25	-1	-1	-1	1	1
26	1	-1	-1	1	1
27	-1	1	-1	1	1
28	1	1	-1	1	1
29	-1	-1	1	1	1
30	1	-1	1	1	1
31	-1	1	1	1	1
32	1	1	1	1	1

$$E = ABCD$$

Exp.	A	B	C	D	E
1	-1	-1	-1	-1	1
2	1	-1	-1	-1	-1
3	-1	1	-1	-1	-1
4	1	1	-1	-1	1
5	-1	-1	1	-1	-1
6	1	-1	1	-1	1
7	-1	1	1	-1	1
8	1	1	1	-1	-1
9	-1	-1	-1	1	-1
10	1	-1	-1	1	1
11	-1	1	-1	1	1
12	1	1	-1	1	-1
13	-1	-1	1	1	1
14	1	-1	1	1	-1
15	-1	1	1	1	-1
16	1	1	1	1	1

Confounding Structure

- Main effects (A, B, C, D, E) are confounded with **4-factor interactions** (e.g., $E = ABCD$).
- Two-factor interactions are confounded with **other 3-factor interactions** (e.g., $AE = BCD$).
- Confounding ensures that higher-order interactions (usually negligible) **do not distort** main effect estimates.
- Significant interactions between three or more factors are **rare**.
- Most models are sufficient when they consider:
 - Main effects.
 - Two-factor interactions.

Resolution of the Design

- The **resolution** of a Design is the sum of the order of effects confounded with one another:
 - Main effects confounded with 4-factor interactions: $4 + 1 = 5$.
 - Two-factor interactions confounded with 3-factor interactions: $3 + 2 = 5$.
- Hence, this is a **Resolution V** design.

Example

- We need 2 generators to go from 16 to 4 experiments. **Resolution is III.**

Prod.	A	B	C	D	E
1	-1	-1	-1	-1	-1
2	1	-1	-1	-1	-1
3	-1	1	-1	-1	-1
4	1	1	-1	-1	-1
5	-1	-1	1	-1	-1
6	1	-1	1	-1	-1
7	-1	1	1	-1	-1
8	1	1	1	-1	-1
9	-1	-1	-1	1	-1
10	1	-1	-1	1	-1
11	-1	1	-1	1	-1
12	1	1	-1	1	-1
13	-1	-1	1	1	-1
14	1	-1	1	1	-1
15	-1	1	1	1	-1
16	1	1	1	1	-1
17	-1	-1	-1	-1	1
18	1	-1	-1	-1	1
19	-1	1	-1	-1	1
20	1	1	-1	-1	1
21	-1	-1	1	-1	1
22	1	-1	1	-1	1
23	-1	1	1	-1	1
24	1	1	1	-1	1
25	-1	-1	-1	1	1
26	1	-1	-1	1	1
27	-1	1	-1	1	1
28	1	1	-1	1	1
29	-1	-1	1	1	1
30	1	-1	1	1	1
31	-1	1	1	1	1
32	1	1	1	1	1

 $D = AB$
 $E = AC$

Exp.	A	B	C	D	E
1	-1	-1	-1	1	1
2	1	-1	-1	-1	-1
3	-1	1	-1	-1	1
4	1	1	-1	1	-1
5	-1	-1	1	1	-1
6	1	-1	1	-1	1
7	-1	1	1	-1	-1
8	1	1	1	1	1

Confusion

Exp.	A	B	C	D	E	AB	AC	AD	AE	BC	BD	BE	CD	CE	DE
1	-1	-1	-1	1	1	1	1	-1	-1	1	-1	-1	-1	-1	1
2	1	-1	-1	-1	-1	-1	-1	-1	-1	1	1	1	1	1	1
3	-1	1	-1	-1	1	-1	1	1	-1	-1	-1	1	1	-1	-1
4	1	1	-1	1	-1	1	-1	1	-1	-1	1	-1	-1	1	-1
5	-1	-1	1	1	-1	1	-1	-1	1	-1	-1	1	1	-1	-1
6	1	-1	1	-1	1	-1	1	-1	1	-1	1	-1	-1	1	-1
7	-1	1	1	-1	-1	-1	-1	1	1	1	-1	-1	-1	-1	1
8	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Aliases

The following main effects/interactions are aliased:

- $A = BD \leftrightarrow D = AB \leftrightarrow DB = AB^2 = A$
- $A = CE \leftrightarrow E = AC \leftrightarrow CE = AC^2 = A$
- $B = AD$
- $C = AE$
- $D = AB$
- $E = AC$
- $BC = DE$
- $BE = CD$

Resolution

- There are **tables** to know the resolution of a design with factors with two levels based on the number of experiments (rows) and number of factors (columns).
- Resolution** is generally presented with Roman numerals

	Factores													
Corr	2	3	4	5	6	7	8	9	10	11	12	13	14	15
4	Com	III												
8		Com	IV	III	III	III								
16			Com	V	IV	IV	IV	III	III	III	III	III	III	III
32				Com	VI	IV	IV	IV	IV	IV	IV	IV	IV	IV
64					Com	VII	V	IV	IV	IV	IV	IV	IV	IV
128						Com	VIII	VI	V	V	IV	IV	IV	IV

Pros and cons of Fractional Factorial Designs (FD)

Pros of FD

- **Resource Efficiency**
 - Requires fewer runs than a full factorial design.
- **Focus on Key Effects**
 - Prioritizes main effects and low-order interactions.
- **Scalability**
 - Suitable for experiments with many factors.

Cons of FD

- **Confounding Effects**
 - Higher-order interactions may be aliased with main effects or low-order interactions.
- **Limited Resolution**
 - Designs with low resolution may not fully separate effects of interest.
- **Requires Assumptions**
 - Assumes higher-order interactions are negligible.

Plot the effects

Example

Use the FrF2 function from the FrF2 package to generate a fractional factorial design

```
library(FrF2)
# Generate a fractional factorial design with 3 factors and 4 runs
fractional_design <- FrF2(nruns = 4, nfactors = 3,
                          factor.names = c("A", "B", "C"))
fractional_design

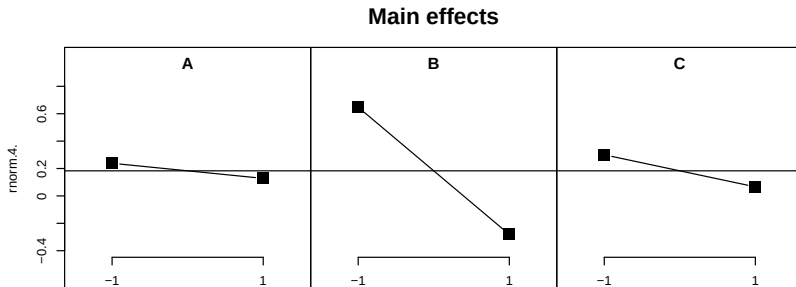
##      A  B  C
## 1 -1 -1  1
## 2  1 -1 -1
## 3 -1  1 -1
## 4  1  1  1
## class=design, type= FrF2
```

```
# Plot main effects
```

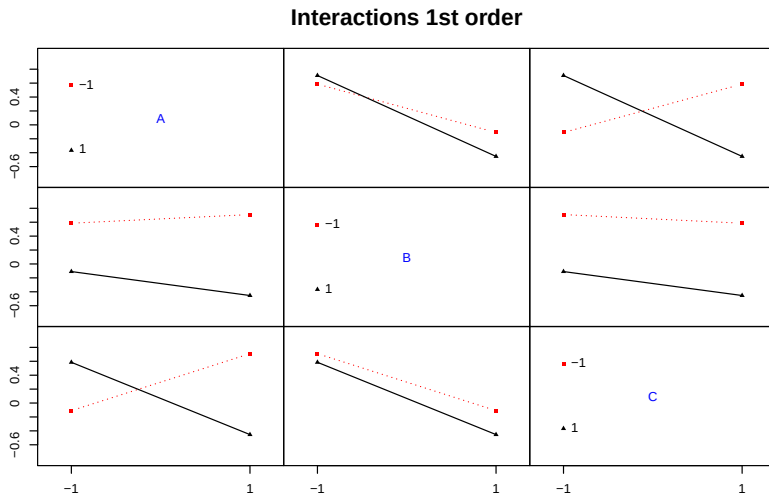
```
set.seed(12345)
```

```
plan <- add.response(fractional_design, response = rnorm(4))
```

```
MEPlot(plan, main = "Main effects")
```



```
IAPlot(plan, main = "Interactions 1st order")
```



Other Experimental Designs

Why Explore Other Designs?

Beyond full and fractional factorial designs, specialized designs address unique experimental challenges:

- **Blocking Factors:** Controlling variability due to nuisance factors.
- **High Dimensionality:** Efficient designs for a large number of factors.
- **Non-Factorial Structures:** Handling irregular or constrained factor levels.

Latin Square Designs

Full Factorial Design

- The FFD requires K^p experiments (i.e., combinations).
- **Example:** Bleach Testing
 - Factors
 - **Brand:** B1, B2, B3
 - **Conditioning:** Classic, Micro, Liquid
 - **Price:** Cheap, Medium, Expensive
 - Full factorial design: $3^3 = 27$ combinations.

Latin Square Design (LS)

- The full design can be reduced to **9** combinations using a **Latin Square**.
- Internal values are obtained by shifting the columns (or rows) one position upwards (or to the left).

Type/Brand	B1	B2	B3
Classic	Cheap	Medium	Expensive
Micro	Medium	Expensive	Cheap
Liquid	Expensive	Cheap	Medium

Latin Square Designs with nuisance factors

Latin Square with nuisance factors

- **Purpose:** LS could be used to reduce variability when several nuisance factors need to be controlled.
- **Structure:** A square matrix where each row and column represents a blocking factor, and treatments are assigned once per row and column.
- **Example:**

A	B	C
B	C	A
C	A	B

Each treatment (A, B, C) appears exactly once in each row and column.

Example

Use the DoE.base R package to generate a Latin Square design:

```
# Create a 3x3 Latin Square design
latin_square <- oa.design(nruns = 9, nfactors = 3, nlevels = 3,
                          factor.names = c("Row", "Column", "Treatment"))
latin_square
```

```
##   Row Column Treatment
## 1    3      2         1
## 2    1      2         3
## 3    2      3         1
## 4    1      3         2
## 5    3      3         3
## 6    1      1         1
## 7    3      1         2
## 8    2      2         2
## 9    2      1         3
## class=design, type= oa
```

Optimal Designs

Optimal Designs

- **Purpose:** Find the most efficient design for a given experimental setup.
- Minimizes the average variance of parameter estimates.
- Useful for experiments with constraints or **irregular factor levels**.

Variance in Linear Models

The variance of the estimates in a linear model is given by:

$$V(\mathbf{b}) = \sigma^2 \cdot (\mathbf{X}'\mathbf{X})^{-1} \quad (\mathbf{X}' \text{ is the transpose of } \mathbf{X})$$

- Since σ^2 is constant, the goal is to minimize:

$$(\mathbf{X}'\mathbf{X})^{-1}$$

- Which implies maximizing:

$$(\mathbf{X}'\mathbf{X})$$

- In other words, seek the maximum determinant.

Example

Use the `optFederov` function from the `AlgDesign` R package to create a D-Optimal Design

```
library(AlgDesign)
# Define factors and levels
factors <- data.frame(Factor1 = rep(c("Low", "Medium", "High"),
                                   each = 2),
                      Factor2 = c("A", "B", "A", "B", "A", "B"))
# Generate a D-optimal design
d_optimal <- optFederov(data = factors, nTrials = 4)
d_optimal$design
```

```
##   Factor1 Factor2
## 1     Low      A
## 2     Low      B
## 3  Medium      A
## 5    High      A
```

Rules of thumbs for choosing the best design

- Choosing the right design depends on the problem's constraints and goals.

Select the design

- **FFD**: No budget constraint.
- **FD**: Easy reduction of FFD.
- **LS Designs**: Effective for controlling two nuisance factors.
- **Optimal Designs**: Tailored for irregular experimental setups to maximize efficiency.

Statistical Analysis in DOE

What is Statistical Importance?

Statistical importance in DOE quantifies the significance of factors and their interactions:

- Determines whether changes in factors lead to meaningful variations in the response.
- Helps prioritize which factors have the most impact on the outcome.

How is Statistical Importance Assessed?

- **ANOVA (Analysis of Variance):**
 - Tests the null hypothesis that a factor has no effect on the response.
 - Provides p-values and F-statistics to evaluate factor significance.
- **Effect Size:**
 - Measures the magnitude of a factor's influence on the response.
 - Example: Partial η^2 (proportion of variance explained).
- **Pareto Charts:**
 - Visualize factor effects ordered by importance.

ANOVA

- **Problem:** Evaluate the impact of batch size and learning rate on model accuracy.
- **Solution:**
 - Significant factors ($p < 0.05$) indicate important effects.
 - Interaction terms reveal combined effects.

```
anova_res <- aov(accuracy ~ batch_size * learning_rate, data = data)
summary(anova_res)
```

```
##                Df    Sum Sq  Mean Sq F value    Pr(>F)
## batch_size      1 0.000579 0.000579    0.555 0.48994
## learning_rate   1 0.028605 0.028605   27.421 0.00336 **
## batch_size:learning_rate 1 0.000000 0.000000    0.000 1.00000
## Residuals      5 0.005216 0.001043
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Visualizing Statistical Importance

- **Pareto Charts:**

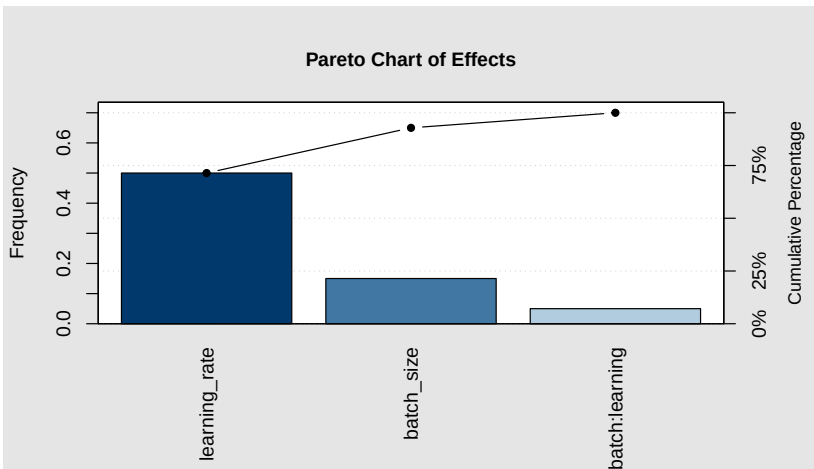
- Highlight the relative importance of factors.
- Example in R using qcc package:

- **Main Effects Plots:**

- Show the direct impact of each factor on the response.

```
library(qcc)
model <- lm(accuracy ~ batch_size * learning_rate, data = data)
effects <- abs(coef(model))[-1]
```

```
pareto.chart(effects, main = "Pareto Chart of Effects")
```



References

- ① Montgomery, D. C. (2017). Design and analysis of experiments (9th ed.). Wiley. ISBN: 978-1119144285
- ② Box, G. E. P., Hunter, W. G., & Hunter, J. S. (2005). Statistics for experimenters: Design, innovation, and discovery (2nd ed.). Wiley-Interscience. ISBN: 978-0471703136